

Конспект 05 — Кэш-память

Содержание

1. Понятие кэш-памяти
2. Организация кэша
3. Устройство данных в кэше
4. Когерентность кэша

”Вам нужно активно говорить: «Мы, конечно, можем, да, но это будет неэффективно!»”

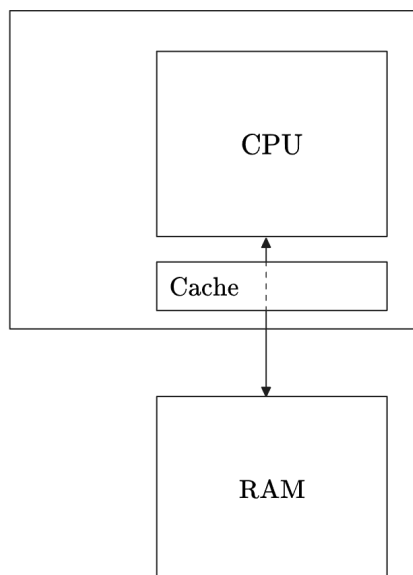
— П. С. Скаков

Понятие кэш-памяти

Всё, как обычно, начинается с проблемы.

У оперативной памяти есть неприятная особенность: она размещена далеко от процессора, в связи с чем скорость доступа к ней печально невелика. Тогда как арифметические операции могут занимать порядка одного такта процессора, операция доступа к памяти растягивается на сотни, и доля времени, которое будут занимать сколь угодно содержательные вычисления в действии уровня «сложить два числа из памяти», будет совершенно крошечной. Конечно, такое положение дел нас не устраивает. Об одном из возможных решений этой проблемы далее и пойдёт речь.

Идея в следующем. Разместим на кристалле процессора небольшой, но очень быстрый с точки зрения скорости доступа буфер, который и назовём **кэшем**:



Кэш может копировать участки оперативной памяти, в результате чего в некоторых случаях при обращении к данным мы можем не идти за ними к ОЗУ, а взять их близлежащую копию, получив ответ гораздо быстрее. Кэш прекрасен в том числе своей практической незаметностью: с точки зрения интерфейса мы почти всегда будем обращаться напрямую к оперативной памяти и не делаем никаких дополнительных действий, получая при этом приятный прирост в производительности.

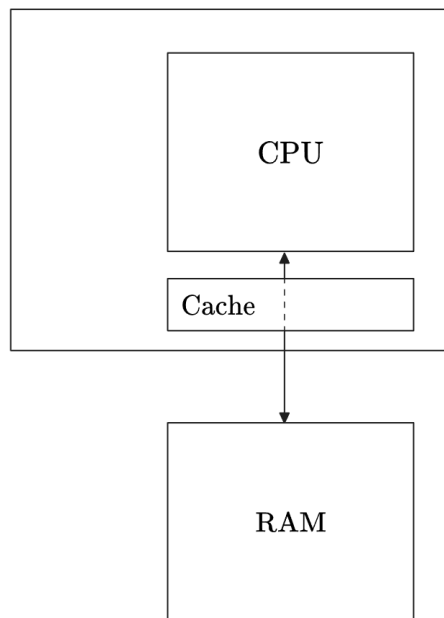
Здесь сразу следует заметить, что кэш рассчитан на многократное обращение к одним и тем же или, по меньшей мере, близко расположенным данным. Он содержит информацию, которая в будущем будет запрошена с высокой вероятностью, но эта вероятность никогда не сто. Например, работу с динамически выделяемыми связными списками или деревьями, то есть с очень неоднородно размещенными в памяти объектами, кэш никак не ускорит.

Алгоритмы, ускоряемые кэшем, можно описать как обладающие свойством *пространственно-временной локальности*. Это значит, что в близкие моменты времени набор данных, с которыми алгоритм работает активно, не сильно изменяется. Это не подразумевает, что данные не изменяются совсем. Например, кэшем очень хорошо ускоряются последовательные вычисления над элементами массива: хотя входные данные алгоритма будут меняться, основной набор локальных переменных, с которыми происходит взаимодействие, остается одним и тем же. Это ощутимо влияет на константы в сложности алгоритмов: например, кэш ощутимо ускоряет быструю сортировку, но практически не ускоряет пирамидальную, и с этим в том числе связана эффективность первой.

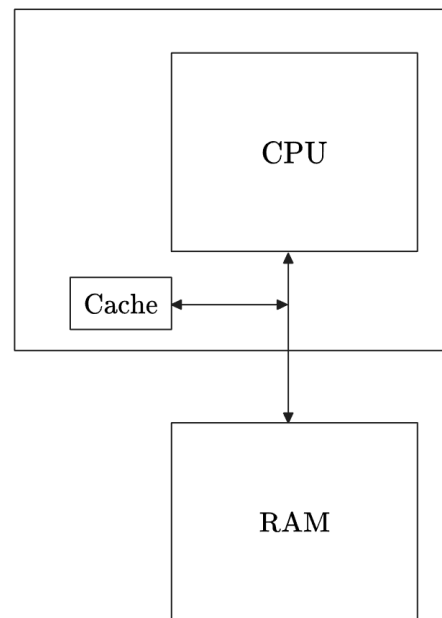
Организация кэша

С точки зрения чтения кэш можно организовать двумя способами:

Look Through Cache



Look Aside Cache



В случае организации look-aside процессор, кэш и оперативная память соединены общей шиной. Эта схема может работать быстрее, когда попадания в кэш не очень частые, и нам не нужно тратить лишнее время на проверку наличия данных в нём. Эта схема также более технически проста в реализации, ибо не требует практически никакого усложнения исходной логики.

В случае же look-through процессор взаимодействует только с кэшем, который сам отправляет запросы ОЗУ. Хотя в пользу look-aside был приведён аргумент, касающийся скорости работы в случае отсутствия данных в кэше, на самом деле look-through существенно быстрее, ибо процент попаданий в кэш на практике очень велик и часто превышает 90%, так что оптимизировать систему под промахи не имеет особенного смысла. Более того, если шина взаимодействия между кэшем и памятью — это дело памяти, то вот шина между кэшем и процессором — это дело сугубо наше, и мы можем ее расширить, полагаясь на высокую частоту нахождения данных в кэше. Вместе с этим look-through технически сложнее, ибо требует отдельного интерфейса взаимодействия между процессором и кэшем, между кэшем и ОЗУ, а также усложнения архитектуры самого кэша.

Режим же записи — это не столько про физическое размещение модуля кэша, сколько про то, как мы манипулируем данными. Здесь также можно выделить два основных подхода:

Write Through

Данные пишутся как в память, так и в кэш.

Write Back

Данные пишутся только в кэш, попадая в память лишь в момент вытеснения.

Разница между этими подходами особенно ощущается при множественной записи: например, в парадигме `write-through` мы будем записывать счётчик цикла в память на каждой итерации, в то время как при работе в режиме `write-back` счётчик будет жить в кэше, пока цикл не закончится, в результате чего мы единожды отдадим последнее его значение памяти.

Какие есть аргументы в пользу каждого из них?

`Write-through` проще. Такой кэш всегда хранит копию оперативной памяти, и нам не нужно никак дополнительно за ними следить. Более того, он лучше с точки зрения защиты от радиации, что полезно при разработке военных и космических технологий.

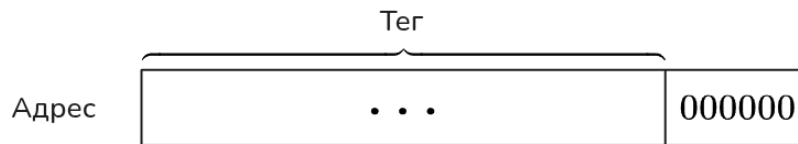
`Write-back` — гораздо более хитрая система, поскольку такой кэш может содержать данные, которых в оперативной памяти ещё нет. Вместе с этим он, конечно, быстрее. Хотя поначалу не до конца понятно, чем именно плохо часто писать в оперативную память: в отличие от операции чтения, мы просто можем отправить запрос на запись и продолжить выполнение как ни в чём не бывало. Дело здесь в том, что у памяти есть очередь команд, которая имеет свойство переполняться, и вот в этот-то момент наши процессы начнут замедляться. Нагрузку на эту очередь `write-back` ощутимо снижает, что ускоряет не только запись, но и чтение, поскольку теперь всем операциям стало одинаково свободно. Помимо прочего, он также способствует и уменьшению энергопотребления, ибо передача данных на достаточно большие расстояния — дорогое развлечение.

Устройство данных в кэше

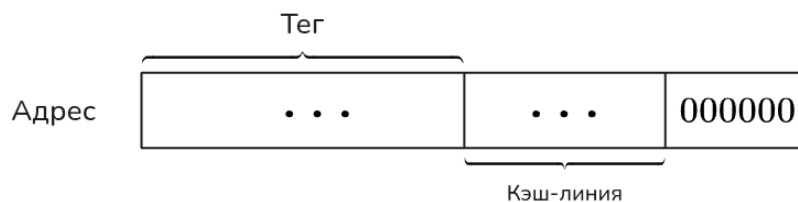
Кэш сохраняет данные не побайтово, а существенно большими порциями, — например, по 64 байта, — которые называются *кэш-линиями*. Причиной этому преимущественно является необходимость сохранения адреса скопированных данных, размер которого ощутимо больше одного байта, в связи с чем нам хотелось бы, чтобы эта информация занимала хотя бы не большую часть данных в кэше.

Такое ограничение наиболее ощутимо влияет на процесс записи в кэш. Дело в том, что объем записываемых данных редко превышает несколько десятков байт, а такое количество информации нельзя записать в кэш физически. По этой причине запрос на запись прежде всего провоцирует запрос на чтение кэш-линии, в которой содержатся данные, и лишь после этого происходит их сохранение.

Чтобы кэш-линии не перекрывались (а они могли бы перекрываться), адрес начала кэшируемой области обязан быть кратен их размеру. Более того, такое ограничение уменьшает количество служебной информации, которую необходимо хранить, ибо последние шесть бит адреса всегда будут нулевыми. Оставшиеся полезные биты адреса будем называть *тегом*.



Кэш-память разбита на уровни, каждый из которых вмещает разное количество кэш-линий. Например, самый быстрый и по совместительству самый маленький L1-кэш обычно имеет объём в районе 32КБ, то есть вмещает 512 64-байтных кэш-линий. Картина эта, честно говоря, радует мало: мы хотели ускорить доступ к памяти, а пришли к необходимости каждый раз проверять полтысячи значений на совпадение. Решить эту проблему можно так: выделим ещё 9 бит адреса под кодирование номера кэш-линии, которая будет содержать наши данные:



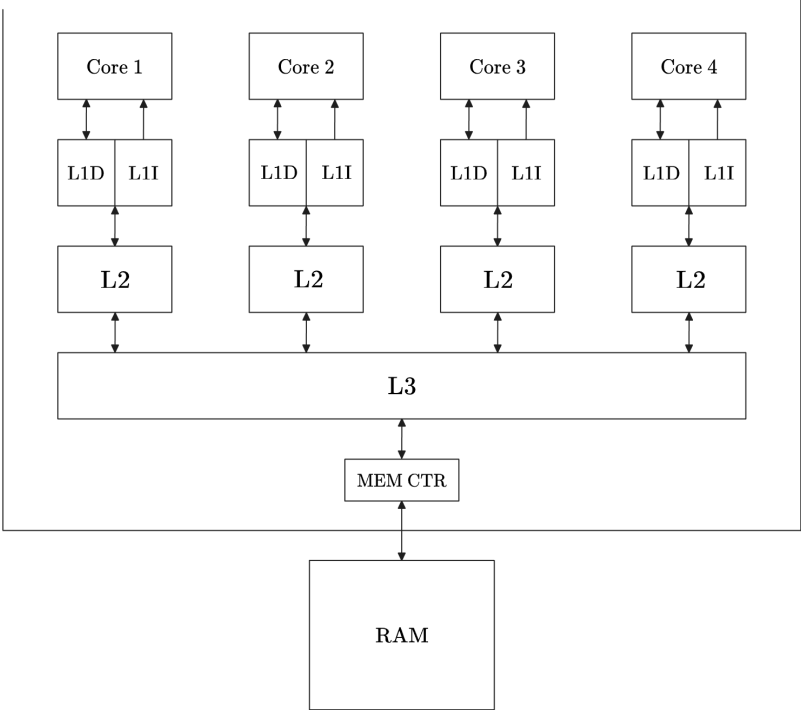
Это называется *прямым отображением*. Оно позволяет не искать номер кэш-линии, а вычислять его мгновенно; более того, оно ещё и сокращает тег. Тем не менее, у такого подхода есть ясная проблема: номера кэш-линий могут совпасть. Если мы возьмем две переменные, адреса которых отличаются только тегом, и будем активно с ними работать, то мы не сможем захэшировать их одновременно, даже если остальные 511 кэш-линий ничем не заняты.

По этой причине на практике используется нечто среднее между непосредственным поиском и прямым отображением. Те же самые 32КБ могут быть организованы в виде 128 групп по 4 кэш-линии, а 7 содержательных битов адреса отведены под кодирование номера группы, внутри которой уже происходит честный полный поиск. Количество линий в такой группе называется *ассоциативностью* кэша.

Продолжая разговор об уровнях кэш-памяти, нельзя не затронуть то, какой вообще вид она имеет в современных процессорах. В случае четырех вычислительных ядер схема

примерно следующая:

Схема организации
кэш-памяти



Как видно, уровней кэш-памяти в современных процессорах три. Это вызвано тем, что мы, с одной стороны, хотим более быстрый кэш, а с другой — более объёмный. Эти желания друг другу противоречат, и всё же мы нашли какой-никакой выход в разбиении на уровни, что позволяет нам иметь маленький, но быстрый кэш одновременно с большим, но медленным. Что касается конкретных характеристик, то показатели примерно следующие:

L1	Объём 32КБ	Скорость доступа 4 такта	Ассоциативность 4
L2	Объём 256КБ	Скорость доступа 14 тактов	Ассоциативность 8
L3 (LLC)	Объём 2МБ × кол-во ядер	Скорость доступа 45 тактов	Ассоциативность 16

Следует сказать, что ассоциативность не обязана возрастать вместе с возрастанием уровня, ровно как и быть степенью двойки, а вот отношение размера кэша к ассоциативности степенью двойки быть скорее обязано — вспоминаем, что хочется получать номер группы из адреса.

Взаимодействие между уровнями кэша определяет понятие *инклюзивности*.

В инклюзивном кэше более высокие уровни всегда дублируют содержимое более низких, что упрощает поиск данных между ядрами, но снижает эффективность использования пространства. При этом они не копируют данные явно, а просто резервируют место и сохраняют ссылку на реальное место их хранения.

В эксклюзивном кэше ситуация ровно обратная: данные никогда не дублируются, поиск дольше, но пространство утилизируется лучше. Существует также неинклюзивный кэш, который не регулирует, дублируются данные или нет, стараясь совместить преимущества обоих подходов.

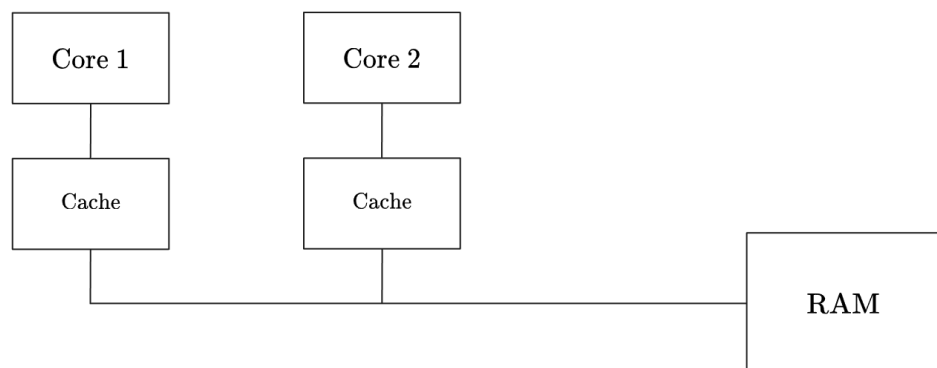
Из схемы также видно, что кэш первого уровня нередко специализирован: у нас есть отдельный кэш L1D для данных и отдельный кэш L1I для команд. О том, зачем конкретно это нужно, мы узнаем, когда будем говорить об устройстве процессора.

Чего из схемы не видно, так это того, что L3 кэш — на практике не совсем монолитная конструкция. Это скорее несколько блоков кэш-памяти, каждый из которых относится к своему ядру, объединённых общей кольцевой шиной, к которой помимо прочего подключены дополнительные устройства вроде контроллера памяти или встроенной видеокарты. Для интегрированной графики, кстати, имеет место добавление ещё одного уровня кэша — L4 — который представлен отдельным кристаллом и подключается параллельно с L3, а не последовательно, и нужен он для ускорения скорости передачи.

Когерентность кэша

Решив с помощью кэша одну проблему, мы сталкиваемся с другой.

Представим систему с двумя вычислителями, — например, двумя ядрами, — оба из которых имеют доступ к оперативной памяти:



Пусть в памяти есть некоторая переменная a , изначально равная нулю. Пусть первый вычислитель выполняет следующий код:

```
while (a == 0) {  
    // ...  
}  
a = 2;
```

а второй — следующий:

```
a = 1;  
while (a == 1) {  
    // ...  
}
```

В таком случае вполне вероятно, что оба цикла будут выполняться бесконечно. Первый поток увидит, что $a = 0$, зайдёт в цикл и будет там себе крутиться, каждый раз проверяя новое значение a из своего кэша. Второй же поток установит $a = 1$ и точно также будет бесконечно долго итерироваться в цикле, сверяясь со своей локальной копией переменной a . Это, конечно, абсурд.

Эта проблема называется проблемой **когерентности** кэша, то есть вопроса о том, чтобы данные об одной области памяти были синхронизированы в различных кэшах.

Первое из допустимых решений — программное — подразумевает вызов команды сброса кэш-линий. Принцип следующий: вызываем сброс *после записи и перед чтением* общих данных. Плохо такое решение в первую очередь своей сложностью: программисты не очень хотят этим заниматься. Ещё одна его проблема в том, что оно пессимистичное: мы сбрасываем кэш всегда, когда обращение из другого потока хотя бы теоретически возможно, а не когда происходит ли оно на самом деле.

Как можно догадаться, второе решение — аппаратное, и почти всегда оно реалистично, то есть сброс происходит лишь тогда, когда это действительно нужно.

Производительность тем самым, конечно, повышается. Добиться этого можно разными способами, основные из которых мы сейчас и рассмотрим.

Протоколы когерентности называются по состояниям, в которых может находиться кэш-линия. Так, в первом из протоколов, которые мы рассмотрим, — MSI, — выделяются следующие состояния:

Modified

В кэш-линии находятся изменённые данные, ещё не записанные в оперативную память

Shared

В кэш-линии находится копия данных из оперативной памяти

Invalid

Кэш-линия пуста

Разберёмся с тем, какие между этими состояниями есть переходы.

Пусть линия находилась в состоянии Invalid. В этом случае кэш будет отвечать на команды следующим образом:

PR (запрос чтения от вычислителя)

Кэш отправляет **BR** (*Bus Read*) и переходит в состояние Shared.

PW (запрос записи от вычислителя)

Кэш отправляет **BRfo** (*Bus Read for ownership*), который полностью передаёт владение данными ему, и переходит в состояние Modified.

BR / BU / BRfo

Кэш остаётся в состоянии Invalid.

Из состояния Shared переходы имеют следующий вид:

PR

Кэш остаётся в состоянии Shared.

PW

Кэш переходит в состояние Modified, отправляя **BU** (*Bus Upgrade*), которая говорит окружающим кэшам, что данные в них неактуальны.

BR

Кэш остаётся в состоянии Shared. Данные можно вернуть прямо из кэша, а можно перенаправить запрос контроллеру памяти — это не влияет на корректность, но может повлиять на производительность.

BU

Кэш переходит в состояние Invalid.

BRfo

Кэш переходит в состояние Invalid.
Данные можно вернуть прямо из кэша, а можно перенаправить запрос контроллеру памяти — это не влияет на корректность, но может повлиять на производительность.

Из состояния Modified переходы имеют следующий вид:

PR / PW

Кэш остаётся в состоянии Modified.

BR

Кэш переходит в состояние Shared/
При этом данные не просто возвращаются, но еще и обязательно записываются в оперативную память.

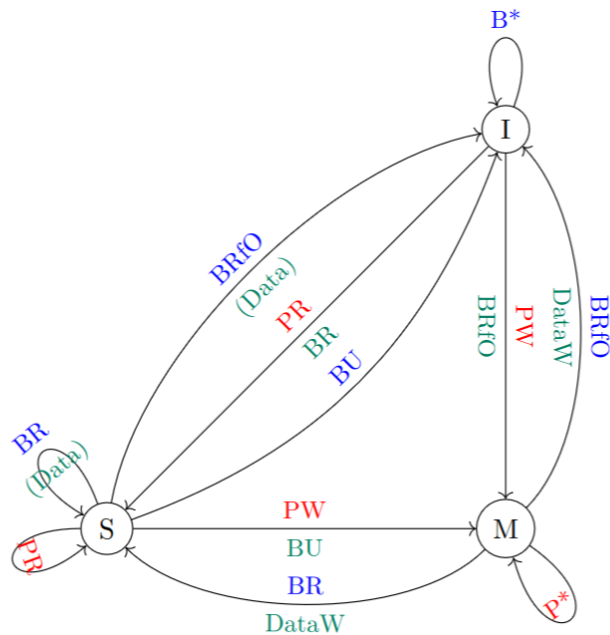
BU

Невозможный переход.

BRfo

Кэш переходит в состояние Invalid.
При этом данные не просто возвращаются, но еще и обязательно записываются в оперативную память.

Всё это можно выразить в виде схемы (*любезно позаимствованной, как и все следующие, из чужого конспекта, ибо автор не горит желанием её перерисовывать*):

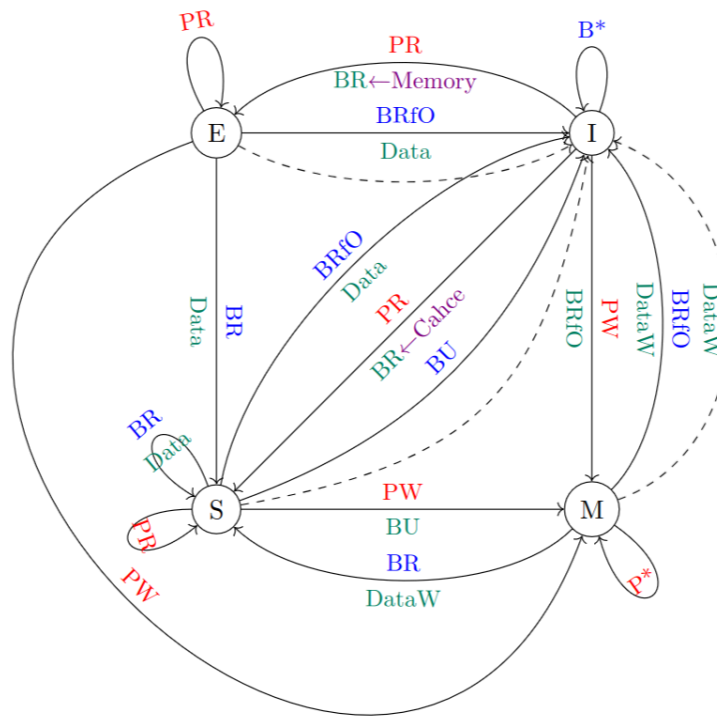


MSI — самый простой протокол когерентности. Он вполне рабочий, но на практике применяется не очень широко в связи с единственным узким местом в производительности — переходом **PW** → **BU**. Большая часть кэшируемых данных на самом деле ни с кем не разделяется, то есть почти всегда глобальное оповещение об обновлении данных не влечет никакого эффекта.

Эту ситуацию улучшает протокол MESI, который добавляет ещё одно состояние Exclusive, — вариацию на тему Shared, — которое кодирует уникальную копию оперативной памяти. Перейти в него можно по команде **PR** в том случае, когда на **BR** отвечает память; если же на него отвечает кэш, то мы переходим в Shared. Вместе с этим кэш в состоянии Shared стал обязан возвращать данные самостоятельно, ибо нам стало принципиально, откуда они приходят. Из самого же состояния Exclusive появились следующие переходы:

- PR** Кэш остаётся в состоянии Exclusive.
- PW** Кэш переходит в состояние Modified.
- BR** Кэш переходит в состояние Shared и отдаёт свои данные.
- BU** Невозможный переход.

Схема протокола MESI выглядит следующим образом:



Можно заметить, что на ней появились пунктирные линии, которые отсутствовали на предыдущей — они отвечают за вытеснение кэш-линии. Легко видеть, что из Exclusive и Shared мы просто переходим в Invalid, а при переходе из Modified мы также должны записать наши данные в память.

Протокол MESI вполне используется, но и он не является пределом совершенства. Когда кто-то запрашивает данные в состоянии Shared, то на последующий запрос **BR** дружно ответят все кэши, которые на эти данные смотрят.

Эта проблема решается в протоколе MESIF добавлением состояния Forwarded — новой вариации на тему Shared. Когда кэшу в состоянии Invalid приходит запрос **PR**, на который отвечает кэш, мы будем переводить его именно в состояние Forwarded, а не Shared. Переходы из него следующие:

PR

Кэш остаётся в состоянии Forwarded.

PW

Кэш переходит в состояние Modified, отправляя **BU**.

BR

Кэш переходит в состояние Shared и отдаёт свои данные.

BU

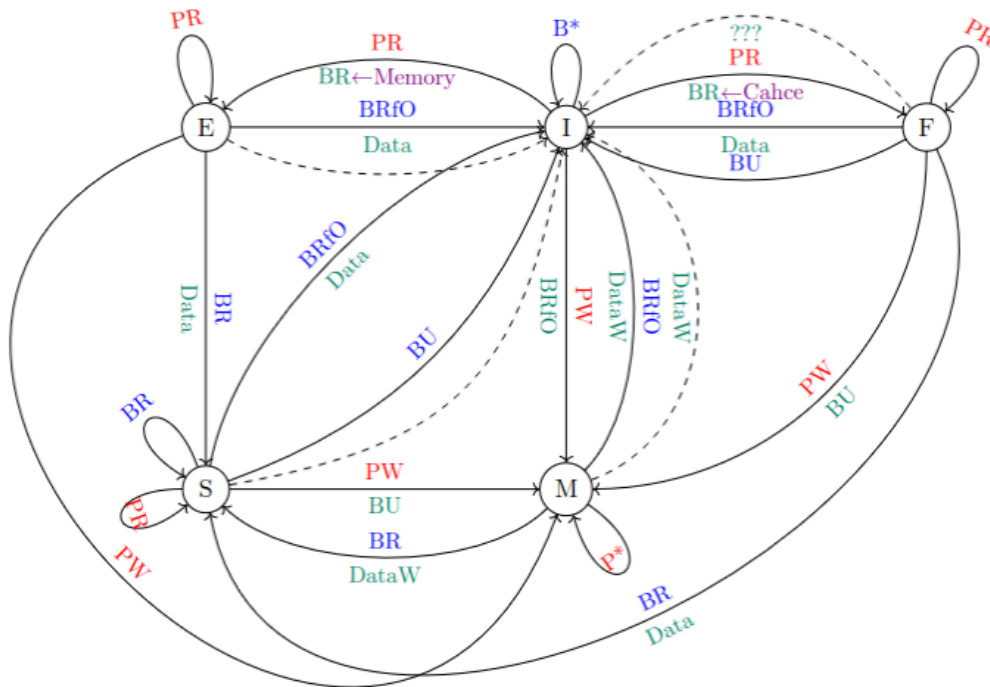
Кэш переходит в состояние Invalid.

BRfo

Кэш переходит в состояние Invalid и отдаёт свои данные.

Суть в том, что Forwarded кэш — это один из Shared кэшей, на которого возложена ответственность по возврату данных по шине. Как только кто-то хочет прочитать его данные, этот «кто-то» становится новым Forwarded, а старый Forwarded переходит в Shared.

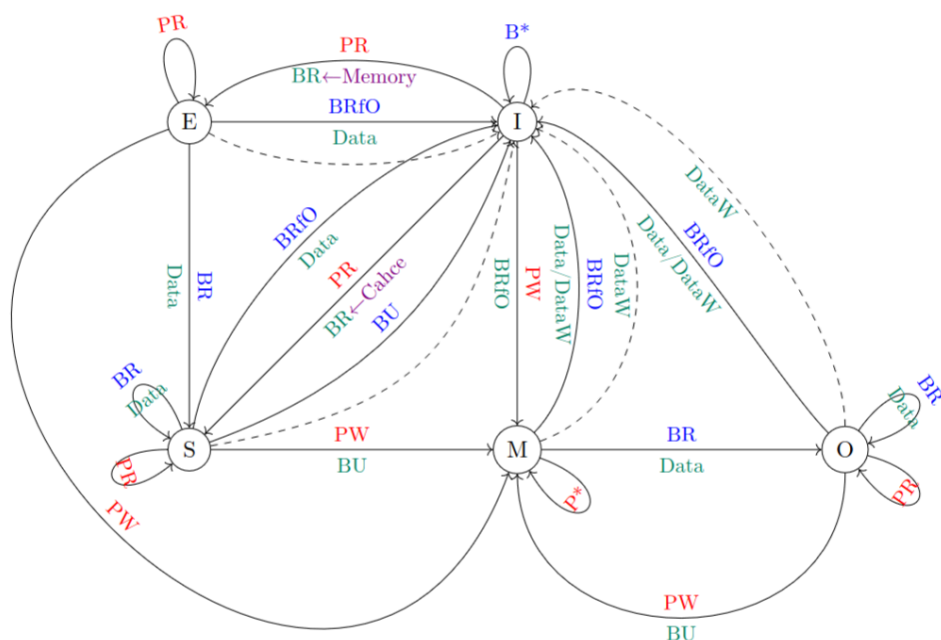
Схема протокола MESIF выглядит следующим образом:



Проблема этого протокола в том, что Forwarded кэш могут выселить, и нам не до конца ясно, что делать в этой ситуации — кэш переназначает нового Forwarded при следующем запросе, но доподлинно неизвестно, как именно.

Другое расширение протокола MESI носит название MOESI и превносит состояние Owner — вариацию на тему Modified, которая возникает из него в результате чтения. При этом Modified «делится» с Owner данными, но не пишет их при этом в память. Поведение этого

состояния практически такое же, как у самого Modified; **PW** переводит его обратно в Modified, вызывая **BU**. Схема этого протокола следующая:



Источники

1. Скаков П. С. — Лекции по аппаратному обеспечению вычислительных систем, 2 семестр, 2026